# Title

## Informations sur le papier

- Titre : Programmation C++
- Auteurs : `Etienne KOA`

## Aperçu de la section

- Variables et constantes
- Déclarer des variables
- Types primitifs `C++`
  - entier
  - virgule flottante
  - booléen
  - personnage
- Opérateur `sizeof`.
- Qu'est-ce qu'une constante ?
- Déclarer des constantes
- Constantes littérales
- Expressions constantes

## Qu'est-ce qu'une variable ?

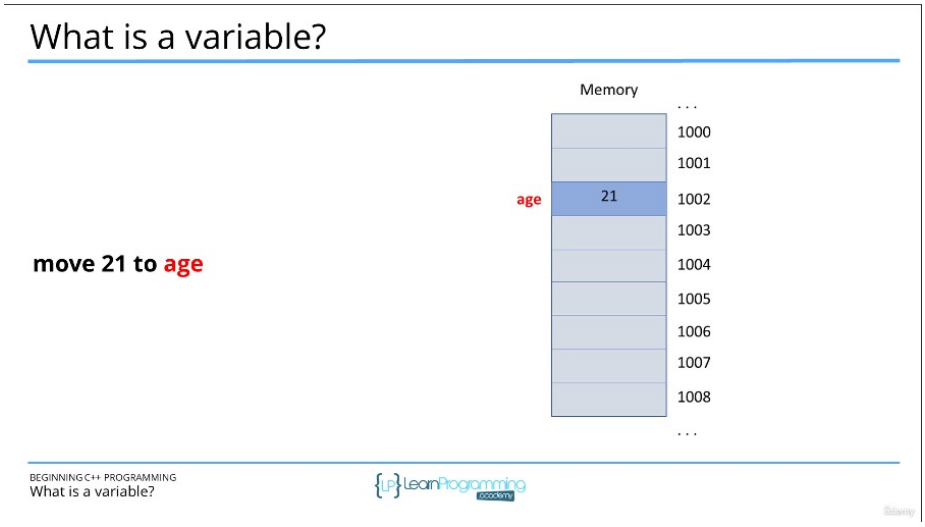


Figure 1: Variable

- Une `variable` est une abstraction pour un emplacement mémoire.
- Autoriser les programmeurs à utiliser des noms significatifs et non des

adresses mémoire.

- Les variables ont
  - `Type` - leur catégorie (integer, real number, string, Person, Account ...)
  - `Valeur` - le contenu (10, 3.14, "Franck"...)
- Les variables « doivent » être déclarées avant d'être utilisées.
- La valeur d'une variable peut changer.

```
age = 21;          // Compiler error
```

```
int age;

age = 21;
```

Figure 2: Variable

## Déclaration et initialisation des variables

- Déclaration de variables

```
VariableType VariableName;
```

```
int age;
double rate;
string name;

Account franks_account;
Person james;
```

Figure 3: Déclaration des variables

- Nommer les variables
  - Peut contenir des lettres, des chiffres et des traits de soulignement.
  - Doit commencer par une lettre ou un trait de soulignement `(_)`
    - Impossible de commencer par un chiffre

  - Impossible d'utiliser les mots-clés réservés `C++`.
  - Impossible de redéclarer un nom dans la même portée.
    * rappelez-vous que `C++` est sensible à la casse.

| Legal | Illegal |
|---|---|
| Age | int |
| age | $age |
| _age | 2014_age |
| My_age | My age |
| your_age_in_2014 | Age+1 |
| INT | cout |
| Int | return |

Figure 4: Nommer les variables

- Nommer les variables - Style et meilleures pratiques.
- Soyez cohérent avec vos conventions de dénomination.
  - `myVariableName` contre `my_variable_name`
  - évitez de commencer les noms par des traits de soulignement.
- Utilisez des noms significatifs
  - ni trop long ni trop court
- N'utilisez jamais de variables avant de les initialiser.
- Déclarez les variables proches du moment où vous en avez besoin dans votre code.
- Initialisation des variables

## Types de données primitifs C++

- Types de données fondamentaux implémentés directement par le langage `C++`.
  - types de caractères
  - types entiers
    * `signé` et `non signé`
  - Types à virgule flottante
  - Type booléen
  - `Size` et `precision` dépendent souvent du compilateur
    * `#include<climits>`

```
int age;   // uninitialized
```

```
int age = 21; // C-like initialization

int age (21); // Constructor initialization

int age {21}; // C++11 list initialization syntax
```

BEGINNING C++ PROGRAMMING
Declaring Variables

Figure 5: Initialisation des variables

- Tailles de caractères
  - Exprimé en bits
  - Plus il y a de bits, plus il y a de valeurs pouvant être représentées.
  - Plus il y a de bits, plus il faut de stockage.

| Size (in bits) | Representable Values | |
|---|---|---|
| 8 | 256 | $2^8$ |
| 16 | 65,536 | $2^{16}$ |
| 32 | 4,294,967,296 | $2^{32}$ |
| 64 | 18,446,744,073,709,551,615 | $2^{64}$ |

Figure 6: Taille du texte

- Types de caractères
  - Utilisé pour représenter des caractères uniques, « A », « X », « @ ».
  - Les types plus sauvages sont utilisés pour représenter des jeux de caractères larges.

| Type Name | Size / Precision |
|---|---|
| **char** | Exactly one byte. At least 8 bits. |
| **char16_t** | At least 16 bits. |
| **char32_t** | At least 32 bits. |
| **wchar_t** | Can represent the largest available character set. |

Figure 7: Types de caractères

- Types entiers

| Type Name | Size / Precision |
|---|---|
| *signed* **short** *int* | At least 16 bits. |
| *signed* **int** | At least 16 bits. |
| *signed* **long** *int* | At least 32 bits. |
| *signed* **long long** *int* | At least 64 bits |

| Type Name | Size / Precision |
|---|---|
| **unsigned short** *int* | At least 16 bits. |
| **unsigned** *int* | At least 16 bits. |
| **unsigned long** *int* | At least 32 bits. |
| **unsigned long long** *int* | At least 64 bits |

Figure 8: Types entiers

- Type à virgule flottante
  - Utilisé pour représenter des nombres non entiers.
  - Représenté par mantisse et exposant (notation scientifique)
  - La précision est le nombre de chiffres de la mantisse.
  - La précision et la taille dépendent du compilateur.

| Type Name | Size / Typical Precision | Typical Range |
|---|---|---|
| float | / 7 decimal digits | $1.2 \times 10^{-38}$ to $3.4 \times 10^{38}$ |
| double | No less than float / 15 decimal digits | $2.2 \times 10^{-308}$ to $1.8 \times 10^{308}$ |
| long double | No less than double / 19 decimal digits | $3.3 \times 10^{-4932}$ to $1.2 \times 10^{4932}$ |

Figure 9: Type à virgule flottante

- Type booléen
  - Utilisé pour représenter le vrai et le faux.
  - Zéro est faux.
  - Non nul est vrai.

## Utilisation de l'opérateur `sizeof`

- L'opérateur `sizeof`.
  - détermine la taille en `octets` d'un type ou d'une variable.

| Type Name | Size / Precision |
|---|---|
| **bool** | Usually 8 bits<br>true or false (C++ keywords) |

Figure 10: Type booléen

- Exemples :

```
sizeof(int)
sizeof(double)
sizeof(some_variable)
sizeof some_variable
```

- `<climits>` ou `<cfloat>`
  - Les fichiers d'inclusion `climits` ou `cfloat` contiennent des informations de taille et de précision sur votre implémentation de `C++`.

```
INT_MAX
INT_MIN
LONG_MIN
LONG_MAX
FLT_MIN
FLT_MAX
```

## Qu'est-ce qu'une constante ?

- Comme les variables `C++`
  - Avoir des noms
  - Occuper le stockage
  - Sont généralement tapés

## Type de constantes en C++

- Constantes littérales
- Constantes déclarées
  - mot-clé `const`
- Expressions constantes
  - mot-clé `constexpr`
- Constantes énumérées
  - mot-clé `enum`
- Constantes définies
  - `#définir`
- Le type de constante le plus évident

```
x = 12 ;
y = 1,56 ;
name = "Franck";
middle_initial = 'J';
```

- Constantes littérales entières

```
12 - un entier
12U - un entier non signé
12L - un nombre entier long
12LL - un entier long et long
```

- Constantes littérales « à virgule flottante »

```
12.1 - un doublé
12.1F - un flotteur
12,1 L - un double long
```

- Constantes littérales `Caractère` (codes d'échappement)

```
\n    - newline
\r    - return
\t    - tab
\b    - backspace
\'    - single quote
\"    - double quote
\\    - backslash

cout << "Hello\tthere\nmy friend\n";
Hello       there
my friend
```

Figure 11: Caractère littéral

- Constantes déclarées à l'aide du mot-clé `const`.

```
const double pi {3.1415926};

const int months_in_year {12};

pi = 2.5;     // Compiler error
```

Figure 12: Constantes

```
#define pi 3.1415926
```

**Don't use defined constants in Modern C++**

Figure 13: Constantes